

LANGGRAPH ET MCP — NOTE DE MISE À NIVEAU

Préparation Enedis · 7 septembre 2026

Les deux seules compétences listées comme rédhibitoires que le parcours ne couvre pas. Aucune des deux n'est difficile : ce sont des réponses à des problèmes que vous rencontrez déjà. L'objectif ici est de tenir la conversation, pas de simuler une expertise.

LangGraph

Le problème qu'il résout

Une chaîne LangChain est un enchaînement : A puis B puis C. C'est suffisant pour un RAG simple. Ça ne l'est plus dès qu'un agent doit **revenir en arrière**, réessayer, brancher selon un résultat, ou attendre une validation humaine.

L'AgentExecutor de LangChain gère bien cette boucle, mais comme une boîte noire : on ne pouvait ni l'inspecter, ni la reprendre après incident, ni insérer un point d'arrêt. Pour un système que d'autres devront exploiter — ce qu'Enedis demande explicitement, « accompagnés pour une reprise en maintenance » — c'est disqualifiant.

LangGraph rend cette boucle explicite : un graphe d'états dont vous dessinez vous-même les nœuds et les transitions.

Le modèle mental

Trois objets suffisent à comprendre.

L'état — un dictionnaire typé qui traverse tout le graphe. Chaque nœud le reçoit et renvoie ce qu'il modifie. Les champs peuvent avoir un *réducteur* qui dit comment fusionner : `add_messages` accumule au lieu d'écraser.

Les nœuds — de simples fonctions. Elles prennent l'état, renvoient une mise à jour. Rien de magique.

Les arêtes — les transitions. Fixes (`add_edge`), ou **conditionnelles** (`add_conditional_edges`) : une fonction lit l'état et décide du nœud suivant. C'est ce qui autorise les cycles.

```
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from typing import Annotated, TypedDict
```

```
class Etat(TypedDict):
```

```

messages: Annotated[list, add_messages] # réducteur : accumule
tentatives: int

def agent(etat): ...      # appelle le LLM
def outils(etat): ...     # exécute les appels d'outils

def router(etat):         # arête conditionnelle
    dernier = etat["messages"][-1]
    if dernier.tool_calls and etat["tentatives"] < 5:
        return "outils"
    return END

g = StateGraph(Etat)
g.add_node("agent", agent)
g.add_node("outils", outils)
g.add_edge(START, "agent")
g.add_conditional_edges("agent", router)
g.add_edge("outils", "agent") # le cycle
app = g.compile(checkpointer=...)

```

Les quatre points qui comptent en production

Points de contrôle. `compile(checkpointer=...)` persiste l'état à chaque pas. Un incident n'oblige pas à tout rejouer : on reprend au dernier point. C'est aussi ce qui donne la mémoire conversationnelle entre appels, indexée par un identifiant de fil.

Validation humaine. `interrupt_before=["outils"]` suspend le graphe avant une action sensible et attend une reprise explicite. C'est le mécanisme d'approbation, natif, sans bricolage.

Bornage. Plafond d'itérations, budget de tokens, délai de garde : ils se posent sur le graphe. C'est la réponse directe au risque d'agent en boucle non bornée — et donc à la charge.

Multi-agents. Le motif courant est un superviseur qui route vers des agents spécialisés, chacun étant lui-même un sous-graphe.

Ce que vous pouvez dire

LangGraph, c'est l'état et les transitions rendus explicites, avec persistance et point d'arrêt. Sur des agents que d'autres doivent exploiter, c'est ce qui rend la boucle inspectable et reprenable — et ce qui permet de la borner, ce qui n'est pas un détail quand la charge est le sujet.

MCP — Model Context Protocol

Le problème qu’il résout

Un agent doit accéder à des API internes. Sans standard, chaque intégration est un connecteur sur mesure : $N \text{ outils} \times M \text{ applications}$, et rien de réutilisable. C’est le problème classique d’intégration point à point, dans sa version IA.

MCP normalise l’interface entre un modèle et le monde extérieur. Une source expose ses capacités une fois ; toute application compatible s’y connecte. On passe de $N \times M$ à $N + M$.

Protocole ouvert, publié par Anthropic fin 2024, adopté depuis largement. Il repose sur **JSON-RPC 2.0**.

L’architecture

Trois rôles.

L’hôte — l’application où vit le modèle. **Le client** — un connecteur dans l’hôte, un par serveur. **Le serveur** — le programme qui expose des capacités.

Le serveur expose trois primitives, et la distinction est structurante :

Primitive	Contrôlée par	Usage
Tools	le modèle	actions qu’il décide d’appeler : interroger une API, écrire
Resources	l’application	données mises à disposition : fichiers, enregistrements, schémas
Prompts	l’utilisateur	modèles d’invite réutilisables, invoqués délibérément

Transports : stdio pour un serveur local, HTTP en flux pour un serveur distant.

Négociation des capacités au moment de la connexion : le client demande ce que le serveur sait faire, et **la découverte des outils se fait à l’exécution**. Ajouter un outil ne demande pas de redéployer l’agent.

```
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("reseau-interne")
```

```
@mcp.tool()
def etat_poste(identifiant: str) -> dict:
    """Retourne l'état courant d'un poste source."""
```

```
    return interroger_api_interne(identifiant)

@mcp.resource("referentiel://postes")
def referentiel() -> str:
    """Référentiel des postes, en lecture."""
    return charger_referentiel()
```

Le point de sécurité

Un serveur MCP décrit lui-même ses outils, et cette description entre dans le contexte du modèle. Un serveur malveillant ou compromis peut donc **influencer le comportement de l'agent par sa seule description** — c'est l'injection par description d'outil. D'où : n'exposer que des serveurs maîtrisés, valider les entrées côté serveur, exiger un consentement explicite sur les actions sensibles, et journaliser les appels.

Chez un opérateur de réseau, ce point n'est pas théorique. Il rejoint directement la mention « règles de cybersécurité » de l'annonce.

L'argument à sortir lundi

C'est le meilleur angle des deux sujets, et il n'est pas évident.

L'annonce exige la **réversibilité** des solutions. Or MCP est précisément un levier de réversibilité : en exposant les API internes derrière un protocole ouvert plutôt que via des connecteurs spécifiques à un framework, on peut changer d'orchestrateur, de framework, voire de modèle, **sans réécrire les intégrations**. Chez un opérateur d'infrastructure dont les systèmes vivent des décennies et survivront à plusieurs générations de modèles, c'est un choix d'architecture, pas une commodité.

MCP m'intéresse surtout pour la réversibilité que vous demandez : exposer les API internes derrière un protocole ouvert plutôt que derrière des connecteurs propres à un framework, c'est ce qui permet de changer d'orchestrateur ou de modèle sans refaire les intégrations.

Ce qu'il faut retenir en trois phrases

LangGraph rend explicites l'état et les transitions d'un agent, avec persistance, reprise et point d'arrêt — donc des agents inspectables, bornables et transmissibles.

MCP normalise l'accès du modèle aux outils et aux données, ce qui transforme $N \times M$ intégrations en $N + M$ et rend le socle réversible.

Aucun des deux n'est un obstacle : le premier formalise une boucle d'agent que vous pratiquez, le second remplace des connecteurs que vous écrivez déjà à la main.

Pour aller plus loin dans la journée

- Documentation LangGraph : parcourir « Introduction », « State », puis l'exemple d'agent avec outils. Une heure suffit à voir la mécanique.
- Spécification MCP sur modelcontextprotocol.io : lire l'architecture et la page sur les trois primitives. Trente minutes.
- Si vous avez une heure de plus : monter un serveur MCP minimal avec FastMCP, deux outils factices, et le brancher sur un client. C'est le genre de détail concret qui se raconte bien en entretien.